

SZERVIZPONT

Fejlesztői dokumentáció

Autószerviz-időpontfoglaló webalkalmazás

Készítette:

Décsi Péter

Décsi Richárd

Décsi Tibor

Képzés: Szoftverfejlesztő és tesztelő

2026

Tartalomjegyzék

Tartalomjegyzék.....	2
1. Bevezetés.....	3
2. Fejlesztési környezet	6
3. Választott technológia és indoklása	7
4. Használt kiegészítő technológiák	8
5. Rendszerarchitektúra és projektstruktúra	9
6. Adatbázis tervezése és felépítése	10
7. Kiválasztott kódrészlet részletes magyarázata	11
8. A fő funkciók megvalósítása.....	12
8.1 Regisztráció és belépés.....	12
8.2 Autók kezelése	13
8.3 Időpontfoglalás.....	14
8.4 Adminisztrátori funkciók	14
8.5 A frontend JavaScript működése.....	15
9. Jogosultságkezelés, validáció és hibakezelés.....	16
10. Reszponzív megjelenítés	16
11. Tesztelési dokumentáció	17
11.1 Tesztelési stratégia	17
11.2 Tesztesetek	19
11.3 Hibajegyzék.....	20
11.4 Tesztelési jegyzőkönyv	21
12. Telepítés, indítás és GitHub-leadás.....	23
13. Összefoglalás és továbbfejlesztési lehetőségek.....	23
13.1 Csapatmunka és feladatmegosztás	23
14. Forrásjegyzék	24

1. Bevezetés

A SzervizPont egy autószerviz számára készített webes időpontfoglaló alkalmazás. A témaválasztás alapja egy hétköznapi, könnyen értelmezhető probléma: kisebb szervizekben az időpontok kezelése gyakran telefonon, üzenetben vagy papíron történik. Ez a módszer kis ügyfélszám mellett működhet, de nehezebbé válik a foglalások visszakeresése, az ügyfélhez tartozó autók nyilvántartása, illetve a szervizszolgáltatások és státuszok követése. A projekt célja ennek a folyamatnak egy egyszerű, böngészőből használható rendszerbe történő összefogása volt.

A projektet azért választottuk, mert jól szemlélteti a képzés során tanult alapvető webfejlesztési feladatokat. A rendszerben egyszerre jelenik meg a frontend, a backend és a relációs adatbázis, továbbá megvalósul a regisztráció és bejelentkezés, a jogosultságkezelés, valamint több CRUD művelet. Az alkalmazás így nem egyetlen technológiát mutat be, hanem azt, hogyan kapcsolódik egymáshoz a böngészőben futó felhasználói felület, a szerveroldali C# kód és az adatbázis.

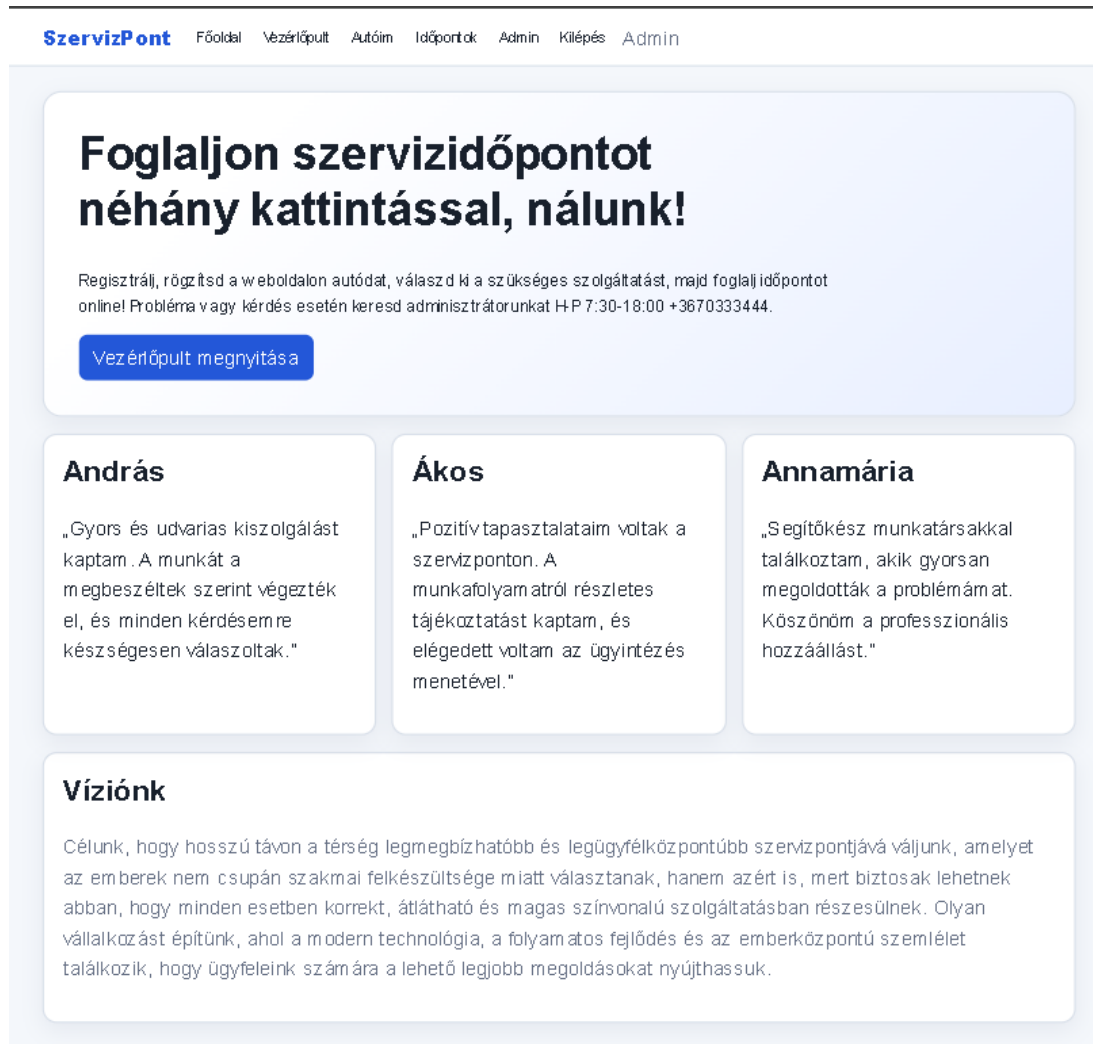
A SzervizPont elsődleges felhasználója az ügyfél. Regisztráció után felveheti a saját autóit, megtekintheti a szolgáltatásokat, időpontot foglalhat, valamint kezelheti a saját foglalásait. A második szerepkör az adminisztrátor, aki a szolgáltatások adatait kezeli, látja a felhasználók listáját és az összes foglalást, továbbá a foglalások státuszát is módosíthatja. A két szerepkör elkülönítése lehetőséget adott arra, hogy a projektben a jogosultságok szerveroldali ellenőrzése is megjelenjen.

A program célközönsége egy kisebb vagy közepes autószerviz és annak ügyfélköre. A vizsgaprojekt nem törekszik egy teljes vállalatirányítási rendszer kiváltására: nem kezel számlázást, raktárkészletet, alkatrészeket, több telephelyet vagy külön szerelői erőforrásokat. A hatókört tudatosan úgy szűkítettük, hogy a fontos webes fejlesztési feladatok jól követhetők és a szóbeli védésen is áttekinthetők maradjanak.

A projekt készítése során a gyakorlati cél az volt, hogy a tanult elméleti fogalmak működő alkalmazássá álljanak össze. Ide tartozik a kliens–szerver kommunikáció, a HTTP-metódusok használata, a JSON-adatok küldése, az adatbázis-táblák közötti kapcsolatok kialakítása, a felhasználói input ellenőrzése és a hibák kezelése. A kész rendszer ezért nem csak bemutató

felület, hanem ténylegesen ment, módosít és töröl adatokat, valamint ellenőrzi a felhasználói jogosultságokat.

Adat	Érték
Projekt neve	SzervizPont
Projekt típusa	Reszponzív webalkalmazás
Frontend	HTML5, CSS3, JavaScript
Backend	ASP.NET Core 8 Web API, C#
Adatbázis	SQLite, Entity Framework Core
Verziókezelés	Git, GitHub
Repository	https://github.com/dezsipeti2000/SzervizPont
Csapattagok	Dézszi Péter, Dézszi Richárd, Dézszi Tibor



1. ábra – A SzervizPont főoldala

A fejlesztés eredményeként egy olyan alkalmazás jött létre, amely helyi környezetben egyszerűen elindítható, mobilon is használható, és GitHub repository-n keresztül átadható. A dokumentáció további része részletesen bemutatja a használt környezetet, a technológiai döntéseket, az adatbázis felépítését, egy kiválasztott kódrészlet működését, a tesztelési folyamatot és a projekt továbbfejlesztési lehetőségeit.

2. Fejlesztési környezet

A fejlesztés Windows alapú számítógépeken történt. A projekt olyan eszközöket használ, amelyek a képzés során tanult webfejlesztési feladatokhoz elegendők, és külön fizetős szerver vagy adatbázis-kezelő szolgáltatás nélkül is futtathatók.

Terület	Használt eszköz / szoftver	Feladat
Operációs rendszer	Windows 10 / Windows 11	Fejlesztési és tesztelési környezet
Kódszerkesztő	Visual Studio Code	HTML, CSS, JavaScript és C# forrásfájlok szerkesztése
Backend futtatókörnyezet	.NET 8 SDK, ASP.NET Core 8	A Web API fordítása és futtatása
Webszerver	ASP.NET Core beépített Kestrel szervere	A helyi HTTP/HTTPS kiszolgálás
Adatbázis	SQLite	Relációs adatok helyi fájlban történő tárolása
ORM	Entity Framework Core	C# objektumok és adatbázis-táblák kezelése
API ellenőrzés	Swagger / OpenAPI	Backend végpontok közvetlen tesztelése
Böngésző	Google Chrome, Microsoft Edge	Felhasználói felület és reszponzivitás ellenőrzése
Verziókezelés	Git és GitHub	Forráskód és dokumentáció közös kezelése
Parancssor	PowerShell	Projekt indítása, Git parancsok és környezet ellenőrzése

A backend futtatásához nincs szükség Apache vagy XAMPP telepítésére, mert az ASP.NET Core saját fejlesztői webszervert használ. Az adatbázis szintén nem igényel külön MySQL szervert, mivel az SQLite egyetlen adatbázisfájlban tárolja az adatokat. Ez egyszerűbbé tette a projekt helyi indítását és átadását.

A fejlesztés során a böngésző fejlesztői eszközeit is használtuk. Ezek segítségével ellenőrizhető a mobil nézet, a hálózati kérések, az esetleges JavaScript hibák és a különböző képernyőméretekhez történő alkalmazkodás. A Swagger felület pedig különösen a backend tesztelésénél volt hasznos, mert a frontend megkerülésével lehetett API-kéréseket küldeni.

3. Választott technológia és indoklása

A feladatkiírás szerint a backend és a frontend technológiája szabadon választható. A gyakran használt HTML–CSS–JavaScript–PHP–MySQL összeállítás helyett a projekt frontendje HTML, CSS és JavaScript, a backendje C# és ASP.NET Core Web API, az adatbázisa pedig SQLite lett.

A frontendhez azért maradtunk a keretrendszer nélküli HTML, CSS és JavaScript használatánál, mert a projekt mérete nem indokolta egy nagyobb kliensoldali keretrendszer bevezetését. Így az oldalak szerkezete, az űrlapok és az eseménykezelés közvetlenül látható a forráskódban, ami kezdő fejlesztőként könnyebben követhető és a védésen is egyszerűbben magyarázható.

A backendnél az ASP.NET Core Web API választását a C# nyelv, a beépített routing, a controllerek, az autentikációs lehetőségek és az Entity Framework Core támogatása indokolta. A Web API jól elkülöníti a szerveroldali logikát a megjelenítéstől: a frontend HTTP-kéréseket küld a controllereknek, amelyek adatbázis-műveleteket végeznek és JSON-választ küldenek vissza.

Az SQLite azért volt megfelelő választás, mert relációs adatbázis, ugyanakkor nem kell hozzá külön adatbázis-szervert telepíteni. Az adatbázis egyetlen fájlban hozható létre, ezért vizsgaprojektnél könnyen hordozható és egyszerűen újraindítható. A projekt négy fő táblája között idegen kulcsokkal kialakított kapcsolatok vannak, tehát teljesíti a relációs adatbázis használatára vonatkozó követelményt.

Réteg	Választott technológia	Miért ezt választottuk?
Frontend	HTML5, CSS3, JavaScript	Egyszerű, jól követhető, külön keretrendszer nélkül is elegendő a feladathoz.
Backend	C#, ASP.NET Core 8 Web API	Controllerek, routing, autentikáció és jól kezelhető szerveroldali logika.
Adatkezelés	Entity Framework Core	A C# modellek és az SQLite közötti adatkezelést egyszerűsíti.
Adatbázis	SQLite	Relációs, könnyen telepíthető, külön szerver nélkül működik.

4. Használt kiegészítő technológiák

1. Entity Framework Core

Az Entity Framework Core egy objektum-relációs leképező eszköz. A projektben az ApplicationDbContext osztály tartalmazza a Users, Cars, Services és Appointments DbSet gyűjteményeket. A controllerek LINQ lekérdezésekkel dolgoznak, az EF Core pedig ezeket SQLite műveletekké alakítja.

2. Cookie-alapú bejelentkezés

A sikeres regisztráció vagy belépés után a backend hitelesítési sütit hoz létre. A szerver a sütiben tárolt claim-ek alapján ismeri fel a felhasználót és a szerepkörét. Az adminisztrátori végpontoknál szerepkör-alapú Authorize ellenőrzés működik.

3. Swagger / OpenAPI

A Swagger automatikusan felsorolja a controllerek API-végpontjait. A Try it out és Execute lehetőségekkel közvetlenül küldhető kérés a backendnek, így ellenőrizhető a JSON kérés, a válasz és a HTTP státusz kód.

4. Fetch API és JSON

A frontend JavaScript fetch függvénnyel kommunikál a backenddel. Az űrlapadatokból JSON készül, a szerver pedig szintén JSON-formátumú adatokat vagy hibaüzenetet küld vissza.

5. Git és GitHub

A projekt repository-ja a forráskódot, az adatbázis-sémát és a dokumentációt egy helyen kezeli. A commitok segítségével követhetővé válnak a módosítások és a csapattagok közreműködése.

6. Reszponzív CSS

A site.css grid elrendezést, tördelhető navigációt, mobil média-lekérdezéseket és vízszintesen görgethető táblázatkeretet tartalmaz, így a felület kisebb képernyőn is használható.

5. Rendszerarchitektúra és projektstruktúra

A SzervizPont kliens–szerver elven működik. A felhasználó a böngészőben a Frontend mappa oldalait használja. A JavaScript az /api útvonalon HTTP-kérést küld az ASP.NET Core backendnek. A controller ellenőrzi a kérést, az Entity Framework Core segítségével lekérdezi vagy módosítja az SQLite adatbázist, majd választ küld vissza a böngészőnek.



A repository felépítése a tanári útmutatásnak megfelelően négy fő mappára van bontva: Frontend, Backend, Database és Documents. A gyökérben található start.bat a program egyszerű indítását, a reset-database.bat pedig a tesztadatbázis törlését segíti.

Mappa / fájl	Tartalom
Frontend/	HTML oldalak, css/site.css és js/*.js fájlok.
Backend/Controllers	Az API végpontok és a szerveroldali műveletek.
Backend/Models	Az adatbázisban tárolt entitások C# osztályai.
Backend/DTOs	Az API-kérések adatátviteli osztályai.
Backend/Data	AppDbContext és az induló adatok létrehozása.
Database/	SQL séma, ER-diagram és futás közbeni SQLite fájl.
Documents/	Fejlesztői és felhasználói dokumentáció.
README.md	A projekt rövid leírása és indítási információk.

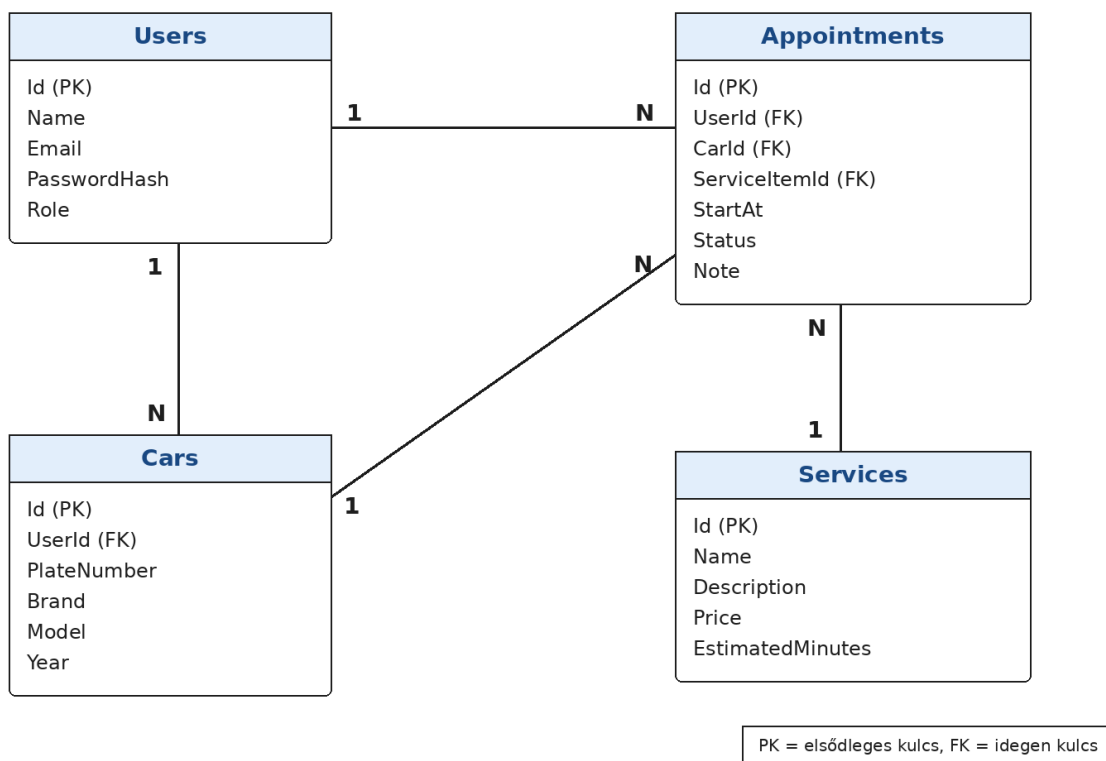
A Program.cs az alkalmazás kiindulópontja. Itt kerül regisztrálásra az adatbázis-kapcsolat, a cookie-alapú hitelesítés, a jogosultságkezelés, a Swagger és a controllerek. A backend a külön Frontend mappát statikus fájlforrásként állítja be, ezért egyetlen ASP.NET Core folyamatból nyitható meg a webes felület és az API.

6. Adatbázis tervezése és felépítése

Az alkalmazás relációs adatbázisa négy fő táblát használ. Minden táblában Id az elsődleges kulcs, a kapcsolódó rekordokat pedig idegen kulcsok kötik össze. Az email cím és a rendszám egyedi indexet kapott, ezért ugyanaz az érték nem rögzíthető kétszer.

Tábla	Fontosabb mezők	Szerepe
Users	Id, Name, Email, PasswordHash, Role	A regisztrált ügyfelek és az adminisztrátor adatai.
Cars	Id, UserId, PlateNumber, Brand, Model, Year	A felhasználóhoz tartozó gépjárművek.
Services	Id, Name, Description, Price, EstimatedMinutes	A foglalható szervizszolgáltatások.
Appointments	Id, UserId, CarId, ServiceItemId, StartAt, Status, Note	Az ügyfél, autó és szolgáltatás közötti foglalás.

SzervizPont adatbázis - táblák és kapcsolatok



2. ábra – A SzervizPont teljes adatbázisának táblái és kapcsolatai

Kapcsolatok: egy User rekordhoz több Car és több Appointment tartozhat; egy Car több Appointment rekordban szerepelhet; egy Service több Appointment rekordhoz kapcsolódhat. Az Appointments tábla tehát egyszerre köti össze a felhasználót, az autót és a szolgáltatást. Ezek mind egy-a-többhöz kapcsolatok.

A Cars.UserId mező biztosítja, hogy minden autó tulajdonosa azonosítható legyen. Az Appointments.UserId, CarId és ServiceItemId mezők az időpont foglalójára, autójára és választott szolgáltatására mutatnak. A kódban törlés előtt külön ellenőrzés történik, hogy kapcsolódó foglalás esetén ne maradjon érvénytelen hivatkozás.

7. Kiválasztott kódrészlet részletes magyarázata

A kiválasztott rész a CarsController új autó létrehozását végző Create metódusa. Ez jól szemlélteti egy teljes backend kérés feldolgozását: adatellenőrzés, jogosultsághoz kötött felhasználóazonosítás, duplikáció ellenőrzése, objektum létrehozása és adatbázisba mentése.

```

01 [HttpPost]
02 public async Task<IActionResult> Create(CarRequest request)
03 {
04     var validationMessage = ValidateCar(request);
05     if (validationMessage != null)
06         return BadRequest(new { message = validationMessage });
07
08     var plateNumber = NormalizePlate(request.PlateNumber);
09     if (await _db.Cars.AnyAsync(car => car.PlateNumber == plateNumber))
10         return BadRequest(new { message = "Ez a rendszám már szerepel a rendszerben." });
11
12     var car = new Car
13     {
14         UserId = GetCurrentUser(),
15         PlateNumber = plateNumber,
16         Brand = request.Brand.Trim(),
17         Model = request.Model.Trim(),
18         Year = request.Year
19     };
20
21     _db.Cars.Add(car);
22     await _db.SaveChangesAsync();
23
24     return Ok(new { message = "Az autó mentve lett." });
25 }

```

Kódrész	Magyarázat
[HttpPost]	A metódus HTTP POST kérést kezel az /api/cars végponton.
CarRequest request	A frontend által elküldött rendszám, márka, modell és évjárat érkezik meg.

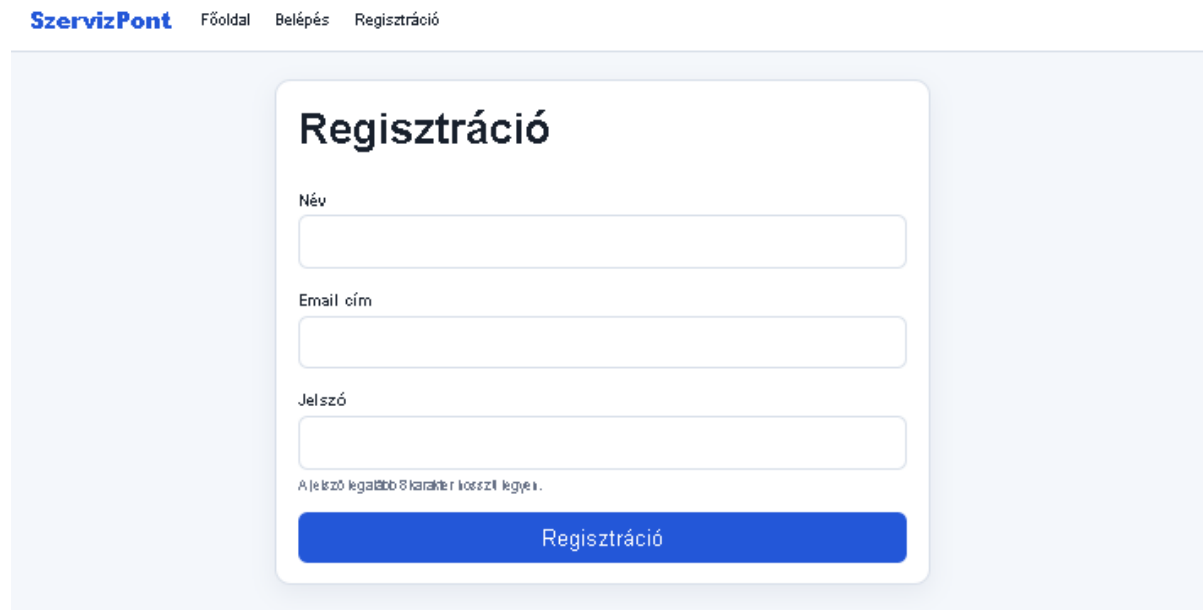
ValidateCar(request)	Ellenőrzi a rendszám formátumát, a kötelező mezőket és az évjáratot.
BadRequest(...)	Hibás adat esetén 400-as választ és magyar nyelvű hibaüzenetet küld.
NormalizePlate(...)	A rendszámot egységes alakra hozza: nagybetűs lesz, a szóköz és kötőjel eltűnik.
AnyAsync(...)	Megnézi, hogy a normalizált rendszám szerepel-e már az adatbázisban.
GetCurrentUserId()	A bejelentkezett felhasználó azonosítóját olvassa ki a hitelesítési adatokból.
new Car { ... }	Létrejön a mentendő Car objektum a feldolgozott adatokkal.
_db.Cars.Add(car)	Az új objektum bekerül az Entity Framework követett elemei közé.
SaveChangesAsync()	Az EF Core ténylegesen elmenti a rekordot az SQLite adatbázisba.
Ok(...)	Siker esetén 200-as válasz és egy visszajelző üzenet érkezik a frontendhez.

A metódus azért is jó példa, mert a felhasználó nem küldi el szabadon a UserId értéket. A tulajdonos azonosítóját a backend a bejelentkezett felhasználóból határozza meg. Így egy ügyfél nem tud egyszerűen egy másik felhasználó azonosítójával autót rögzíteni.

8. A fő funkciók megvalósítása

8.1 Regisztráció és belépés

Az AuthController register végpontja ellenőrzi a nevet, az e-mail formátumát, a jelszó legalább nyolc karakteres hosszát és az e-mail egyediségét. A jelszó az ASP.NET Core beépített PasswordHasher osztályával hash formában kerül tárolásra. Sikeres regisztráció után a rendszer automatikusan be is jelentkezeti a felhasználót.



SzervizPont Főoldal Belépés Regisztráció

Regisztráció

Név

Email cím

Jelszó

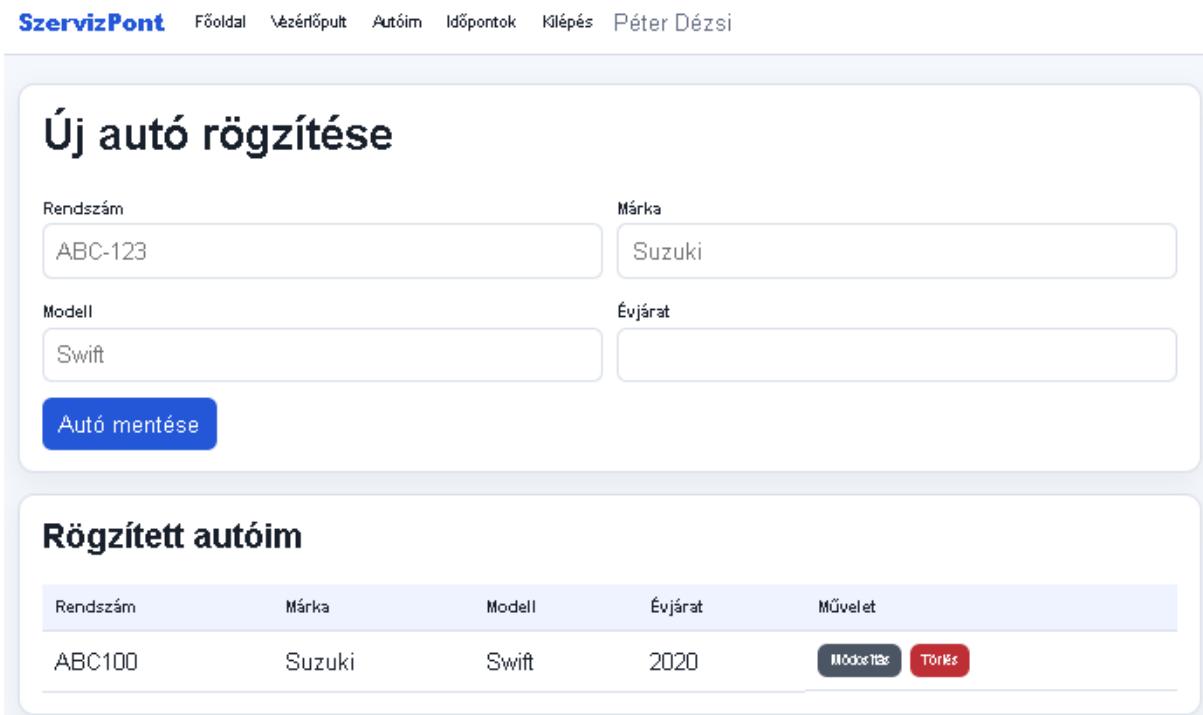
A jelszó legalább 8 karaktert kell legyen.

Regisztráció

3. ábra – Regisztrációs oldal

8.2 Autók kezelése

A CarsController GET, POST, PUT és DELETE műveleteket biztosít. Normál ügyfél csak a saját autóit látja és kezelheti. Adminisztrátor kérés esetén az all paraméterrel az összes autó lekérhető. A rendszám mentés előtt normalizálásra kerül, és az adatbázisban egyedi érték.



SzervizPont Főoldal Vezérlőpult Autóim Időpontok Kilépés Péter Dézsi

Új autó rögzítése

Rendszám

Márka

Modell

Évjárat

Autó mentése

Rögzített autóim

Rendszám	Márka	Modell	Évjárat	Művelet
ABC100	Suzuki	Swift	2020	<button>Módosítás</button> <button>Törölés</button>

4. ábra – Autó rögzítése és módosítása

8.3 Időpontfoglalás

Az ügyfél csak a saját autójához foglalhat időpontot. Kötelező autót és szolgáltatást választani, a kezdésnek legalább 30 perccel későbbinek kell lennie az aktuális időnél. A megjegyzés legfeljebb 500 karakter. Ugyanarra a kezdési időpontra a rendszer nem enged két aktív foglalást; a lemondott időpont nem számít ütközésnek.

SzervizPont Főoldal Vezérlőpult Autóim Időpontok Kilépés Péter Dézsi

Új időpont foglalása

Az időpont rögzítve lett.

Autó: Szolgáltatás:

Kezdési időpont:

Megjegyzés:

Időpontjaim

Időpont	Autó	Szolgáltatás	Státusz	Megjegyzés	Művelet
2026. 09. 28. 13:00:00	ABC100	Fékellenőrzés (45 perc)	Függőben	–	Módosítás Törlés

5. ábra – Sikeresen rögzített időpont és az időpontlista

8.4 Adminisztrátori funkciók

Az adminisztrátor szolgáltatást hozhat létre, módosíthat és törölhet. Látja a felhasználókat és az összes időpontot, továbbá a foglalások státuszát Függőben, Megerősítve, Elvégezve vagy Lemondva értékre állíthatja. A ServicesController írási műveletei és a UsersController csak Admin szerepkörrel érhetők el.

SzervizPont Főoldal Vezérlőpult Autók Időpontok Admin Képek Admin

Adminisztráció

A szolgáltatások, felhasználók és az összes foglalás kezelése.

Új szolgáltatás

Név: Ár (Ft):

Időtartam percben:

Leírás:

Szolgáltatás mentése

Szolgáltatások

Név	Leírás	Ár	Időtartam	Művelet
Fékekellenőrzés	A fékrendszer állapotának átvizsgálása.	15 000 Ft	45 perc	Módosítás Töröl
Olajcsere	Motorolaj és olajszűrő cseréje.	28 000 Ft	60 perc	Módosítás Töröl
Általános diagnosztika	Hibakódolvasás és alapvető műszaki átvizsgálás.	12 000 Ft	30 perc	Módosítás Töröl

Felhasználók

Név	Email	Szerepkör
Admin	admin@szervizpont.hu	Adminisztrátor
Péter Dézsi	dezsipeti2000@gmail.com	Ügyfél

Összes időpont

Időpont	Üg: #1	Autó	Szolgáltatás	Megjegyzés	Szabvány	Művelet
2026. 09. 28. 13:00:00	Péter Dézsi	ABC100	Fékekellenőrzés (45 perc)	–	Függőben	Töröl

6. ábra – Adminisztrátori felület

8.5 A frontend JavaScript működése

Az `api.js` tartalmazza a közös API hívó függvényt, a hibaüzenetek kezelését, a dátum- és pénzformázást, valamint a navigáció beállítását. Az egyes oldalakhoz külön JavaScript-fájl tartozik: `cars.js` az autókat, `appointments.js` az időpontokat, `admin.js` az adminisztrátori funkciókat kezeli.

A `fetch` kérések `credentials`: "same-origin" beállítással futnak, ezért a böngésző a hitelesítési sütit is elküldi a backendnek. 401-es válasz esetén a frontend törli a helyben tárolt egyszerű felhasználói adatot és visszairányít a belépéshez. 403-as válasznál jogosultsági hibaüzenet jelenik meg.

A dinamikusan létrehozott táblázatcellák `textContent` használatával kapják meg a szöveges adatot. Ez egyszerűbb és biztonságosabb, mint a felhasználói értékek HTML-kódként történő beillesztése.

9. Jogosultságkezelés, validáció és hibakezelés

A rendszer két szerepkört használ: User és Admin. Regisztrációkor minden új felhasználó User szerepkört kap. Az adminfiókot a DatabaseSeeder hozza létre az adatbázis első indításakor.

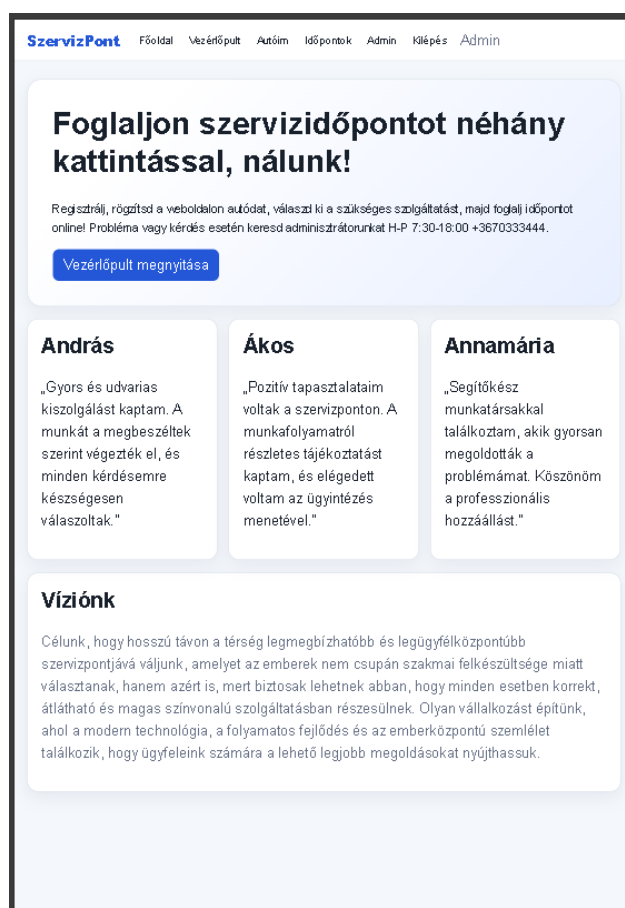
A védett controllereken [Authorize] attribútum található. Az adminisztrátori műveleteknél [Authorize(Roles = Roles.Admin)] szerepel. A frontend ugyan elrejti az Admin menüpontot a normál ügyfél elől, de a valódi védelem a backendben történik. Ha egy ügyfél közvetlenül meghív egy admin végpontot, a szerver 403 Forbidden választ ad.

A jelszavakat a rendszer nem egyszerű szöveggént tárolja. A PasswordHasher hash értéket készít, és belépéskor a keretrendszer a megadott jelszót ehhez hasonlítja. A hitelesítési cookie HttpOnly, ezért a JavaScript nem olvashatja ki közvetlenül.

Ellenőrzés	Példa	Backend válasz
E-mail formátum	hibás cím	400 + „Érvénytelen email cím.”
Jelszó hossza	8 karakternél rövidebb	400 + hibaüzenet
Duplikált e-mail	már regisztrált cím	400 + hibaüzenet
Rendszám	túl rövid / érvénytelen karakter	400 + hibaüzenet
Autó tulajdonjoga	más felhasználó autója	400 vagy 403
Foglalási idő	30 percen belüli kezdés	400 + hibaüzenet
Admin jogosultság	User admin végpontot hív	403 Forbidden
Nem létező rekord	hibás azonosító	404 Not Found

10. Reszponzív megjelenítés

A felhasználói felületnek a vizsgakiírás szerint mobil eszközön is használhatónak kell lennie. A site.css rugalmas grid elrendezést használ, a navigáció tördelhető, a táblázatok pedig .table-wrap elemen belül vízszintesen görgethetők. 800 pixel alatt az űrlapok és kártyák egy oszlopba rendeződnek, 520 pixel alatt pedig további méretigazítások lépnek életbe.



7. ábra – A főoldal mobil szélességen

A mobilteszt során a főoldalt, a navigációt, az űrlapokat és a táblázatokat ellenőriztük. A cél nem egy külön mobilalkalmazás készítése volt, hanem az, hogy ugyanaz a weboldal kisebb kijelzőn is olvasható és kezelhető maradjon.

11. Tesztelési dokumentáció

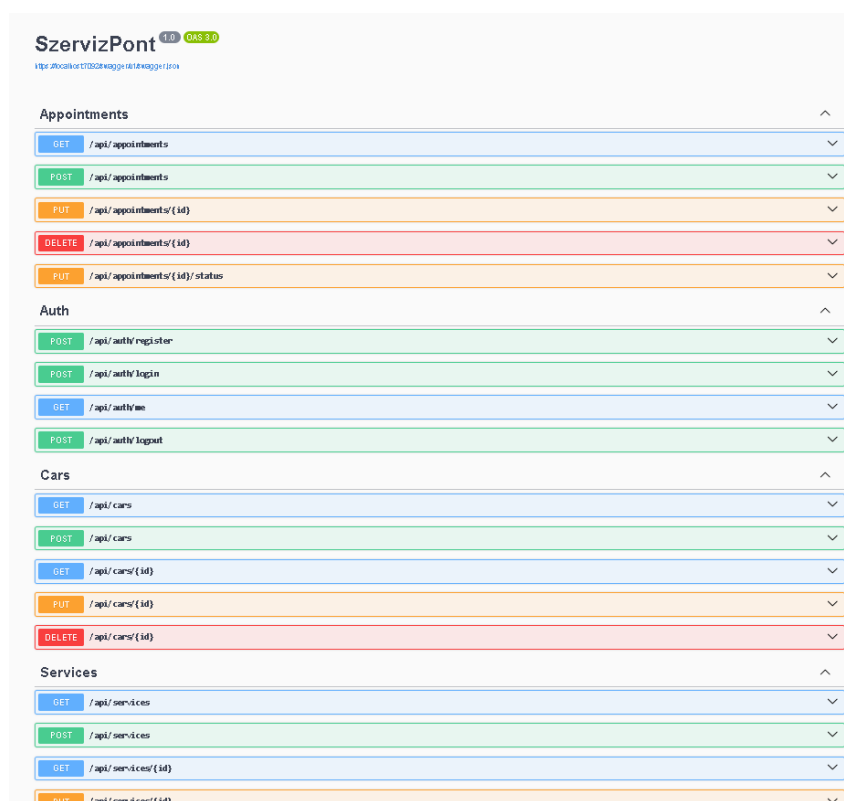
11.1 Tesztelési stratégia

A tesztelés célja annak ellenőrzése volt, hogy a SzervizPont fő funkciói megfelelnek a felhasználói igényeknek, a hibás bemenetek kezelése egyértelmű, az adatbázis-kapcsolatok nem sérülnek, valamint az ügyfél és az adminisztrátor jogosultságai megfelelően elkülönülnek.

Elsősorban manuális funkcionális tesztelést végeztünk. A böngészőből végrehajtottuk a regisztráció, bejelentkezés, autókezelés, időpontfoglalás és adminisztráció fő folyamatait. Emellett Swagger segítségével közvetlenül is meghívtuk az API-végpontokat, így a HTTP-

státusz kódok és a JSON-válaszok is ellenőrizhetők voltak. A mobil megjelenést Chrome/Edge fejlesztői eszközökkel vizsgáltuk.

Tesztkörnyezet	Beállítás
Operációs rendszer	Windows 10/11
Backend	.NET 8, ASP.NET Core 8
Adatbázis	SQLite
Böngésző	Google Chrome, Microsoft Edge
API teszt	Swagger UI
Módszer	Manuális funkcionális, integrációs és jogosultsági teszt



8. ábra – A Swagger által felsorolt API-végpontok

11.2 Tesztesetek

ID	Leírás	Lépések	Elvárt eredmény	Tényleges eredmény	Sikeres?
T001	Helyes regisztráció	Új név, e-mail és legalább 8 karakteres jelszó; Regisztráció	Felhasználó létrejön és belép	200 OK; felhasználó létrejött	Igen
T002	Ismételt e-mail	Már létező e-maillal új regisztráció	Hibaüzenet, nincs új rekord	400; hibaüzenet megjelent	Igen
T003	Hibás jelszavas belépés	Helyes e-mail + hibás jelszó	Belépés sikertelen	401 Unauthorized	Igen
T004	Új autó	Rendszám, márka, modell, évjárat; Mentés	Autó létrejön	200; autó megjelent a listában	Igen
T005	Duplikált rendszám	ABC-123 után ABC123 rögzítése	Duplikáció elutasítása	400; hibaüzenet	Igen
T006	Autó módosítása	Meglévő autó adatainak átírása	Adatok frissülnek	200; módosítás mentve	Igen
T007	Foglalással rendelkező autó törlése	Időponttal rendelkező autó Törlés	Törlés megtagadva	400; kapcsolati hiba megelőzve	Igen
T008	Szolgáltatások lekérése	GET /api/services	Lista visszaérkezik	200 OK	Igen
T009	Helyes időpontfoglalás	Saját autó, szolgáltatás, jövőbeli időpont	Foglalás létrejön Függőben státusszal	200; foglalás megjelent	Igen
T010	Túl közeli időpont	30 percen belüli kezdés	Foglalás elutasítása	400; hibaüzenet	Igen
T011	Azonos kezdési időpont	Már foglalt kezdés ismételt megadása	Második aktív foglalás ne jöjjön létre	400; hibaüzenet	Igen
T012	Ügyfél admin API-t hív	Userként GET /api/users	Hozzáférés megtagadva	403 Forbidden	Igen
T013	Admin szolgáltatást kezel	Létrehozás, módosítás, törlés	Műveletek végrehajthatók	200; műveletek sikeresek	Igen
T014	Státusz módosítása	Admin foglalás státuszát módosítja	Új státusz mentésre kerül	200; státusz frissült	Igen

T015	Kijelentkezés	POST /api/auth/logout, majd védett kérés	Munkamenet megszűnik	200, majd 401 a védett kérésre	Igen
T016	Mobil nézet	Chrome DevTools, 375 px szélesség	Felület kezelhető, nem esik szét	Megfelelő megjelenítés	Igen

11.3 Hibajegyzék

Az alábbi hibajegyzék a fejlesztés és a végső tesztelés során kezelt, a SzervizPont méretéhez reálisan illeszkedő problémákat foglalja össze. Olyan eseteket választottunk, amelyek a kész program működésében is ellenőrizhetők, ezért a védeken megmutatható, hogy a végleges verzió milyen módon kezeli őket.

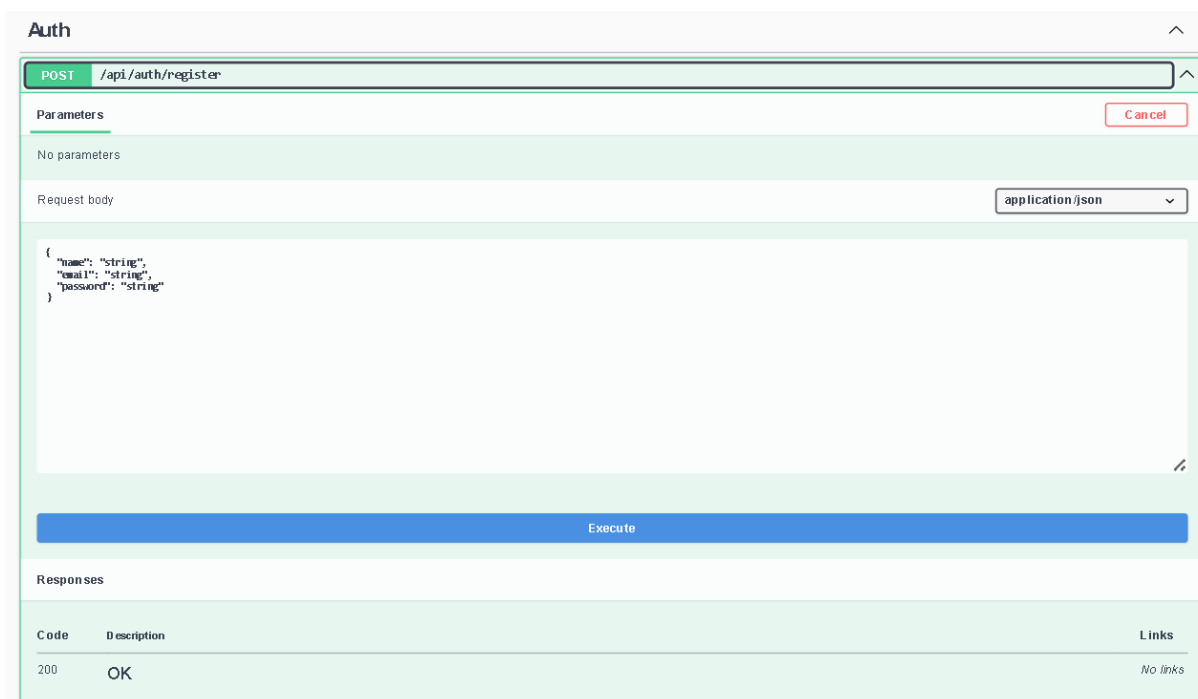
Hibaazonosító	Leírás	Állapot	Javítás / megjegyzés
H001	A különböző formában megadott rendszámok (pl. ABC-123 és ABC123) duplikált autónak számítanak, de az eltérő írásmód miatt korábban külön értéként kezelhetők voltak.	Javítva	NormalizePlate egységesíti a rendszámot, az adatbázisban egyedi index is van.
H002	Jogosultság nélküli API kérésnél a cookie-kezelés alapértelmezésben bejelentkezési oldalra irányíthatott helyett, hogy API-státuszkódot adott volna.	Javítva	OnRedirectToLogin = 401, OnRedirectToAccessDenied = 403 beállítás került a Program.cs-be.
H003	Foglalással rendelkező autó törlése adatkapcsolati problémát vagy hibás hivatkozást okozhatott volna.	Javítva	Törlés előtt AnyAsync ellenőrzi, hogy tartozik-e Appointment az autóhoz.
H004	Ugyanarra a kezdési időpontra több aktív foglalás kerülhetett volna be.	Javítva	HasSameStartTime ellenőrzés megakadályozza a duplikált aktív kezdési időt.
H005	Normál ügyfél kézzel beírhatta az admin.html címet, ezért a felületet is külön védeni kellett.	Javítva	requireAdmin visszairányít; a backend ettől függetlenül szerepkör alapján 403-at ad.
H006	Kisebb képernyőn a széles táblázatok kilóghattak az oldalról.	Javítva	A table-wrap overflow-x:auto beállítást kapott, a grid mobilon egy oszlopra vált.

11.4 Tesztelési jegyzőkönyv

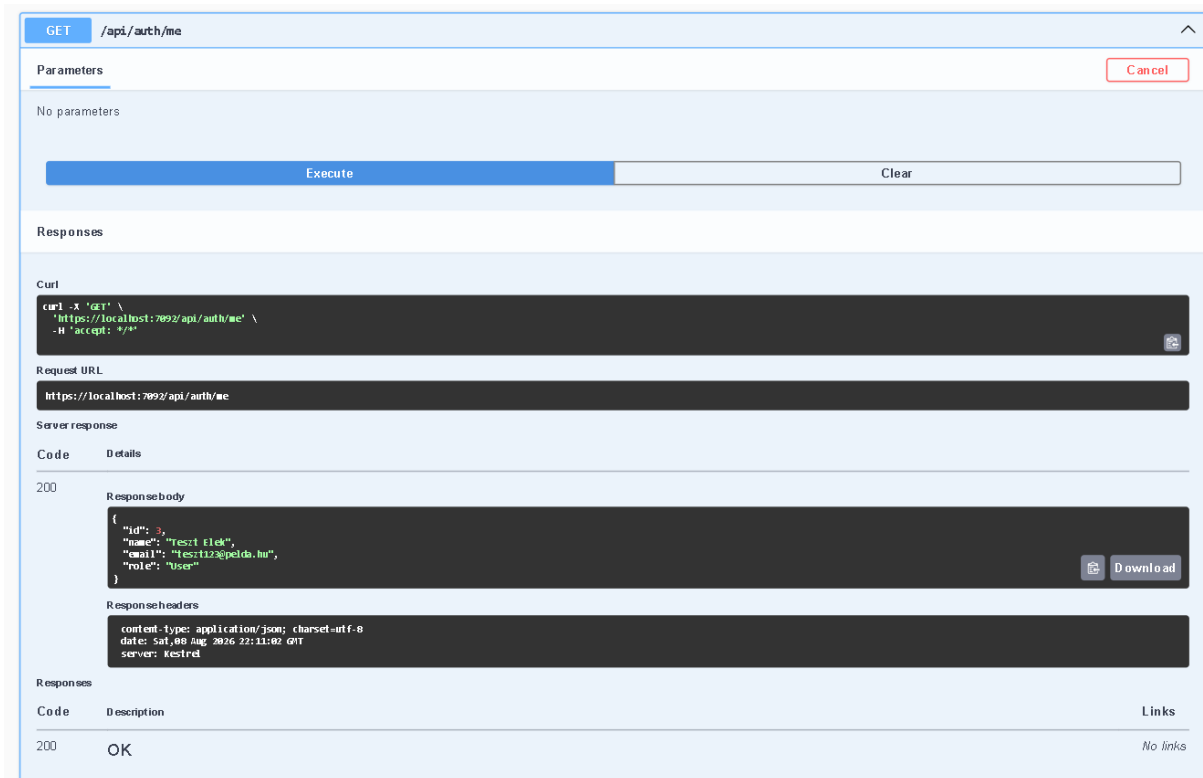
Tesztelők: Dézsi Péter, Dézsi Richárd és Dézsi Tibor. Tibor elsősorban a felhasználói felület manuális végigpróbálásában és a hibák feljegyzésében vett részt; a technikai API- és jogosultsági teszteket Péter és Richárd végezte.

Tesztelés időszaka: 2026. szeptember

Eredmény: a fő felhasználói és adminisztrátori funkciók, a fontos validációk, az adatbázis-kapcsolatok és a jogosultsági esetek ellenőrzésre kerültek. A végleges tesztkörben kritikus nyitott hiba nem maradt. A Swaggerben végzett tesztek során a sikeres műveleteknél 200 OK, a hibás vagy jogosulatlan kéréseknél a tervezett 400/401/403 státuszkódok érkeztek.



9. ábra – Regisztrációs végpont tesztelése Swaggerben



10. ábra – A bejelentkezett felhasználó lekérdezése 200 OK válasszal

A tesztelés során nem csak a „sikeres” útvonalakat vizsgáltuk, hanem szándékosan hibás eseteket is: ismételt e-mail-címet és rendszámot, túl közeli időpontot, azonos kezdési időpontot, illetve jogosultság nélküli admin API-hívást. Ez azért fontos, mert a rendszer helyes működéséhez nem elég, hogy a jó adatok menthetők legyenek; a hibás vagy tiltott műveleteket is következetesen el kell utasítani.

12. Telepítés, indítás és GitHub-leadás

A projekt GitHub repository-n keresztül kerül átadásra. A repository:

<https://github.com/dezsipeti2000/SzervizPont>. A tanári kérésnek megfelelően külön Frontend, Backend, Database és Documents mappa található benne.

Helyi futtatás lépései:

1. A repository letöltése vagy klónozása.
2. A .NET 8 SDK telepítettségének ellenőrzése a `dotnet --list-sdks` paranccsal.
3. A projekt gyökérmappájában a `start.bat` elindítása.
4. A megnyíló böngészőben a localhost cím használata.
5. API teszteléshez a `/swagger` útvonal megnyitása.

A futás közben létrejövő `Database/szervizpont.db` fájl tartalmazza a helyi tesztadatokat. A `reset-database.bat` leállított alkalmazás mellett törli ezt a fájlt, így a következő indításkor a `DatabaseSeeder` új, tiszta adatbázist és alapadatokat hoz létre.

13. Összefoglalás és továbbfejlesztési lehetőségek

A projekt pozitív eredménye, hogy a tervezett fő funkciók egy egységes webalkalmazásban működnek. Elkészült a regisztráció és bejelentkezés, a saját autók kezelése, a szolgáltatásválasztás, az időpontfoglalás, az adminisztrátori szolgáltatás- és státuszkezelés, valamint a reszponzív felület. A frontend, a backend és a relációs adatbázis ténylegesen kommunikál egymással, ezért a projekt teljes folyamatot mutat be a felhasználói kattintástól az adatbázis-műveletig.

A továbbfejlesztés egyik iránya a valódi szervizkapacitás kezelése lehet. Jelenleg a rendszer az azonos kezdési időpontokat tiltja, de később külön szerelőket, munkaállásokat és a szolgáltatások teljes időintervallumát is figyelembe lehetne venni. További lehetőség az e-mailes értesítés, jelszó-visszaállítás, számlázási modul, alkatrészkezelés, több telephely és részletes statisztika.

13.1 Csapatmunka és feladatmegosztás

A projektet háromfős csapatban készítettük, eltérő technikai tapasztalattal. A fejlesztési feladatok súlypontja Péter és Richárd között oszlott meg, míg Tibor főként felhasználói szemléletű manuális teszteléssel, hibák feljegyzésével és a dokumentáció ellenőrzésével segítette a projektet. Ezt a

munkamegosztást azért választottuk, mert így mindenki olyan részfeladatot vállalt, amelyhez a saját tapasztalata és szerepe a legjobban illeszkedett.

Csapattag	Elsődleges feladatok
Dézsai Péter	Frontend oldalak és felhasználói folyamatok kialakítása; HTML/CSS/JavaScript felület; űrlapok és navigáció; felületi ellenőrzések; a felhasználói dokumentáció összeállításának támogatása.
Dézsai Richárd	Backend API és controllerek; adatbázis-kapcsolatok és modellek; autentikáció és jogosultságkezelés; validációk; frontend-backend integráció és technikai hibajavítások.
Dézsai Tibor	Felhasználói szemléletű manuális tesztelés; regisztráció, belépés, autókezelés és időpontfoglalás végigpróbálása; mobil- és asztali nézet ellenőrzése; észlelt hibák feljegyzése; képernyőképek és felhasználói dokumentáció ellenőrzése.
Közös feladat	A követelmények egyeztetése, a kész funkciók közös átnézése, végső tesztelés, dokumentáció ellenőrzése és a leadási állapot kialakítása.

A munkamegosztás nem azt jelenti, hogy minden csapattagnak azonos mélységben kellett programoznia. Tibor közreműködésének értéke elsősorban abban állt, hogy a programot végfelhasználói szemmel próbálta ki, olyan hibás vagy nehezen érthető helyzeteket keresett, amelyek fejlesztés közben könnyen elkerülhetik a programozó figyelmét. A védésen ezért a saját tesztelési és dokumentációs feladatairól tud részletesen beszámolni, míg a mélyebb kód- és adatbázis-kérdéseket elsősorban Péter és Richárd ismerteti.

14. Forrásjegyzék

Microsoft Learn: ASP.NET Core dokumentáció – <https://learn.microsoft.com/aspnet/core/>

Microsoft Learn: Entity Framework Core dokumentáció – <https://learn.microsoft.com/ef/core/>

Microsoft Learn: ASP.NET Core Identity / PasswordHasher – <https://learn.microsoft.com/aspnet/core/security/authentication/identity>

SQLite Documentation – <https://www.sqlite.org/docs.html>

MDN Web Docs: HTML, CSS, JavaScript és Fetch API – <https://developer.mozilla.org/>

Swagger / OpenAPI – <https://swagger.io/docs/>

GitHub Docs: Git és repository kezelés – <https://docs.github.com/>